```
3) #include <stdio.h>
#include <omp.h>
#define N 20
int main()
{
   int fib[N];
   fib[0] = 0;
   fib[1] = 1;
   for (int i = 2; i < N; i++) {
      fib[i] = fib[i - 1] + fib[i - 2];
   }
   printf("Fibonacci sequence (sequential):\n");
   for (int i = 0; i < N; i++) {
      printf("%d ", fib[i]);
   }
   printf("\n");
   #pragma omp unroll
   for (int i = 2; i < N; i++) {
      fib[i] = fib[i - 1] + fib[i - 2];
   }
   printf("Fibonacci sequence (parallel):\n");
   for (int i = 0; i < N; i++) {
      printf("%d ", fib[i]);
   }
   printf("\n");
```

```
    return 0;
}
4) #include <stdio.h>
#include <omp.h>
#define NUM_UNITS 2
#define NUM_INSTRUCTIONS 6
typedef struct {
    char type;
    int src1;
    int src2;
    int dest;
} Instruction;
typedef struct {
    char type;
    int op1;
    int op2;
    int result;
    int status;
} FunctionalUnit;
Instruction instructions[NUM_INSTRUCTIONS] = {
    {'A', 2, 3, 1},
    {'M', 4, 5, 2},
    {'A', 1, 2, 3},
    {'M', 3, 3, 4},
    {'A', 5, 4, 5},
```

```
    {'M', 2, 2, 6}
};
FunctionalUnit units[NUM_UNITS];
void execute_instruction(int unit_id, Instruction inst) {
    for (int i = 0; i < 100000000; ++i);
    if (inst.type == 'A') {
        units[unit_id].result = inst.src1 + inst.src2;
    } else if (inst.type == 'M') {
        units[unit_id].result = inst.src1 * inst.src2;
    }
    units[unit_id].status = 0;
}
int main() {
    for (int i = 0; i < NUM_UNITS; ++i) {
        units[i].status = 0;
    }
    #pragma omp parallel for num_threads(NUM_INSTRUCTIONS)
    for (int i = 0; i < NUM_INSTRUCTIONS; ++i) {
        int unit_id = -1;
        #pragma omp critical
        {
            for (int j = 0; j < NUM_UNITS; ++j) {
                if (units[j].status == 0) {
                    unit_id = j;
                    units[j].status = 1;
```

```
                break;
            }
        }
    }
    if (unit_id != -1) {
        execute_instruction(unit_id, instructions[i]);
        printf("Executed instruction %d: Result = %d\n", i + 1, units[unit_id].result);
    } else {
        printf("No free functional unit available for instruction %d\n", i + 1);
    }
  }
  return 0;
}
```

8)

```
#include <omp.h>
#include <stdio.h>
int a, b, i, tid;
float x;
#pragma omp threadprivate(a, x)
int main() {
  omp_set_dynamic(0);
  printf("1st parallel region:\n");
  #pragma omp parallel private(b, tid)
  {
    tid = omp_get_thread_num();
    a = tid;
```

```
      b = tid;
      x = 1.1 * tid + 1.0;
      printf("thread %d: a,b,x = %d %d %f\n", tid, a, b, x);
   }
   printf("********************\n");
   printf("master thread doing serial work here\n");
   printf("*********************************\n");
   printf("2nd parallel region:\n");
   #pragma omp parallel private(tid)
   {
      tid = omp_get_thread_num();
      printf("thread %d: a,b,x = %d %d %f\n", tid, a, b, x);
   }
   return 0;
}
```

9)

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
int factorial(int n)
{
   int i, out;
   out = 1;
   for (i = 1; i < n + 1; i++)
      out *= i;
   return out;
```

```
}
int main(int argc, char **argv)
{
   int i, j, threads;
   int *x;
   int n = 12;
   if (argc > 1)
   {
      threads = atoi(argv[1]);
      if (omp_get_dynamic())
      {
         omp_set_dynamic(0);
         printf("called omp_set_dynamic(0)\n");
      }
      omp_set_num_threads(threads);
   }
   printf("%d threads\n", omp_get_max_threads());
   x = (int *)malloc(n * sizeof(int));
   for (i = 0; i < n; i++)
      x[i] = factorial(i);
   j = 0;
   #pragma omp parallel for firstprivate(x, j)
   for (i = 1; i < n; i++)
   {
      j += i;
```

```
    x[i] = j * x[i - 1];
  }
  for (i = 0; i < n; i++)
    printf("factorial(%d) = %d   x[%d] = %d\n",
        i, factorial(i), i, x[i]);
  free(x);
  return 0;
}
```

10)

```
#include <stdlib.h>
#include <stdio.h>
#include <omp.h>
#define N 100000000
#define TRUE 1
#define FALSE 0
int main(int argc, char **argv )
{
char host[80];
int *a;
int i, k, threads, pcount;
double t1, t2;
int found;
if (argc > 1)
{
threads = atoi(argv[1]);
if (omp_get_dynamic())
```

```
{
omp_set_dynamic(0);
printf("called omp_set_dynamic(0)\n");
}
omp_set_num_threads(threads);
}
printf("%d threads max\n",omp_get_max_threads());
a = (int *) malloc((N+1) * sizeof(int));
for (i=2;i<=N;i++) a[i] = 1;
k = 2;
t1 = omp_get_wtime();
#pragma omp parallel firstprivate(k) private(i,found)
while (k*k <= N)
{
#pragma omp for
for (i=k*k; i<=N; i+=k) a[i] = 0;
found = FALSE;
for (i=k+1;!found;i++)
{
if (a[i]){ k = i; found = TRUE; }
}
}
t2 = omp_get_wtime();
printf("%.2f seconds\n",t2-t1);
pcount = 0;
```

```
for (i=2;i<=N;i++)

{

if( a[i] )

{

pcount++;

}

}

printf("%d primes between 0 and %d\n",pcount,N);

}
```